

MVC TEMPLATE — HELLO WORLD → TMS

Reproduire l'écran "Chauffeurs" de Truck Management Service

Feuille de route composant par composant — React (View) → NestJS
(Controller) → Service (Model) → Prisma → MySQL. Chaque étape
indique les fichiers à créer ou modifier, avec leur emplacement exact
dans le projet.

Périmètre validé : Chauffeurs (CRUD complet) · Véhicules & Attelages (onglets vides) · Tâches (liées à un Chauffeur) · Tableau via TanStack Table · Style
web sobre

PARTIE 1 — BACKEND : MODEL "CHAUFFEUR"

1 Ajouter le model Prisma Chauffeur MODEL — schéma

On décrit d'abord la forme des données en base, avant tout code applicatif — c'est la fondation dont dépendent le Service et le Controller.

```
backend/  
└─ prisma/  
   └─ schema.prisma (ajouter le model Chauffeur)
```

```
cd backend npx prisma migrate dev --name add_chauffeur
```

2 Module NestJS Chauffeurs (CRUD complet) CONTROLLER MODEL — logique

Même structure que HelloModule (déjà dans le projet) : un Controller qui reçoit l'HTTP, un Service qui parle à Prisma, un Module qui assemble les deux.

```
backend/  
└─ src/  
   └─ chauffeurs/  
      ├── chauffeurs.module.ts (assemble controller + service)  
      ├── chauffeurs.controller.ts (GET / GET:id / POST / PATCH:id / DELETE:id)  
      ├── chauffeurs.service.ts (this.prisma.chauffeur.*)  
      └─ dto/  
         ├── create-chauffeur.dto.ts  
         └── update-chauffeur.dto.ts  
└─ app.module.ts (imports: [..., ChauffeursModule])
```

PARTIE 2 — BACKEND : MODEL "TÂCHE" (liée à un Chauffeur)

3 Model Prisma Tâche + relation vers Chauffeur MODEL — schéma

Une Tâche appartient toujours à un Chauffeur (chauffeurId), avec une date d'échéance, un motif et un statut (à traiter / traité / reporté).

```
backend/  
└─ prisma/  
   └─ schema.prisma (ajouter model Tache { chauffeurId, dueDate, motif, statut })
```

```
npx prisma migrate dev --name add_tache
```

4 Module NestJS Tâches CONTROLLER MODEL — logique

Route dédiée pour lister les tâches à venir et pour changer leur statut (bouton Traité / Reporté du panneau).

```
backend/  
└─ src/  
   └─ taches/  
      ├── taches.module.ts  
      ├── taches.controller.ts (GET / PATCH:id/statut)  
      ├── taches.service.ts  
      └─ dto/  
         └── update-tache-statut.dto.ts  
└─ app.module.ts (imports: [..., TachesModule])
```

PARTIE 3 — FRONTEND : OSSATURE GÉNÉRALE (View)

5 Layout général : bandeau, nav icônes, onglets [VIEW](#)

On sépare la coquille visuelle (toujours affichée) du contenu qui change selon l'onglet actif — App.tsx devient un simple chef d'orchestre.

```
frontend/  
└─ src/  
  └─ layout/  
    ├── AppLayout.tsx (assemble Header + Sidebar + Tabs + contenu)  
    ├── Header.tsx    (bandeau logo)  
    ├── Sidebar.tsx   (icônes verticales, décoratif)  
    ├── Tabs.tsx      (Chauffeurs / Véhicules / Attelages)  
    └─ layout.css  
└─ App.tsx (remplace le contenu actuel par <AppLayout/>)
```

PARTIE 4 — FRONTEND : ONGLET CHAUFFEURS

6 Client API + hook de données [VIEW — accès données](#)

Centraliser les appels fetch vers le Controller NestJS (comme le fetch déjà présent dans App.tsx, mais réutilisable pour chaque entité).

```
frontend/  
└─ src/  
  └─ features/  
    └─ chauffeurs/  
      ├── api.ts          (getChauffeurs, createChauffeur, updateChauffeur, deleteChauffeur)  
      └─ useChauffeurs.ts (hook React : liste + état chargement/erreur)
```

7 Installer et configurer TanStack Table [VIEW](#)

```
cd frontend npm install @tanstack/react-table
```

```
frontend/  
└─ src/  
  └─ features/  
    └─ chauffeurs/  
      ├── columns.ts      (définition des colonnes : Chauffeur, Catégorie, Téléphone, Société, Lien personnel, Archivé)  
      └─ ChauffeursTable.tsx (useReactTable + rendu <table>, tri, cases à cocher)
```

8 Barre d'outils (toolbar) [VIEW](#)

4 groupes de boutons comme dans la capture : Général / Sélection / Edition / Filtre.

```
frontend/  
└─ src/  
  └─ features/  
    └─ chauffeurs/  
      └─ ChauffeursToolbar.tsx  
        (Rafraîchir, Imprimer, Tout sélectionner, Désélectionner,  
         Nouveau, Modifier, Supprimer, case à cocher Archivé)
```

9 Formulaire Nouveau / Modifier [VIEW](#)

Un formulaire modal réutilisé pour la création et l'édition, qui appelle api.ts (POST ou PATCH selon le cas) puis rafraîchit la table.

```
frontend/  
└─ src/  
  └─ features/  
    └─ chauffeurs/  
      └─ ChauffeurForm.tsx
```

10 Assembler la page Chauffeurs [VIEW](#)

```
frontend/  
└─ src/  
  └─ features/  
    └─ chauffeurs/  
      └─ ChauffeursPage.tsx  
        (assemble ChauffeursToolbar + ChauffeursTable + ChauffeurForm)  
  └─ layout/Tabs.tsx (onglet "Chauffeurs" affiche <ChauffeursPage/>)
```

PARTIE 5 — FRONTEND : ONGLETS VIDES + PANNEAU TÂCHES

11 Onglets Véhicules et Attelages (vides) [VIEW](#)

```
frontend/  
└─ src/  
  └─ features/  
    ├── vehicules/  
    │   └─ VehiculesPage.tsx (placeholder "à venir")  
    └─ attelages/  
        └─ AttelagesPage.tsx (placeholder "à venir")  
  └─ layout/Tabs.tsx (branche les 2 nouveaux onglets)
```

12 Panneau latéral Tâches [VIEW](#)

Liste les tâches à venir (GET /taches), regroupées par date, avec les actions Traité / Reporté qui appellent PATCH /taches/:id/statut.

```
frontend/  
└─ src/  
  └─ features/  
    └─ taches/  
      ├── api.ts (getTaches, updateTacheStatut)  
      ├── useTaches.ts  
      ├── TacheCard.tsx (une carte : chauffeur, motif, date, boutons)  
      └─ TachesPanel.tsx (liste triée par date + scroll)  
  └─ layout/AppLayout.tsx (ajoute <TachesPanel/> en colonne de droite)
```

RÉCAPITULATIF — ORDRE D'EXÉCUTION

1. schema.prisma (Chauffeur) + migration
2. chauffeurs/ (module NestJS complet)
3. schema.prisma (Tache) + migration
4. taches/ (module NestJS complet)
5. layout/ (Header, Sidebar, Tabs, AppLayout)
6. features/chauffeurs/api.ts + useChauffeurs.ts
7. features/chauffeurs/columns.ts + ChauffeursTable.tsx
8. features/chauffeurs/ChauffeursToolbar.tsx
9. features/chauffeurs/ChauffeurForm.tsx
10. features/chauffeurs/ChauffeursPage.tsx (assemblage)
11. features/vehicules/ + features/attelages/ (placeholders)
12. features/taches/ (TachesPanel complet)